IntelliTravel Pioneer — AI Agent 应用实现架构介绍

大家好，今天我来介绍 IntelliTravel Pioneer 这款 AI 驱动智能旅行规划平台的 Agent 应用实现架构。内容将涵盖整体架构设计、Agent 的三阶段循环机制、MCP 工具集成、SSE 流式传输、前端 Agent 面板以及智能合并策略。

一、整体架构概览

先来看整体架构。它分为三层：

- **前端**：Vue 3 + TypeScript 构建的单页应用，负责用户交互和 Agent 思考过程的实时展示。
- **后端**：Spring Boot 3 + JPA 的微服务架构，核心 Agent 逻辑实现在 `HttpTripPlanningAiClient` 这个类中。
- **AI 与工具层**：AI 能力依赖 SiliconFlow 平台的 Qwen3-8B 大语言模型，地图数据通过高德 MCP Server 提供的工具获取。

前后端之间通过 SSE（Server-Sent Events）协议进行流式通信。后端的 Agent 在运行过程中，会通过 SseEmitter 实时推送思考日志和生成进度，前端通过原生 fetch API 解析事件流并渲染到 Agent 面板上。

二、Agent 三阶段循环设计

整个行程生成流程被设计为三个有序的阶段，每一阶段都有独立的 System Prompt 和目标。

第一阶段：PLAN（规划阶段）

这个阶段不带工具调用。我们使用 PLAN_SYSTEM_PROMPT 指导大模型进行纯文本推理。模型需要输出三部分内容：需求分析、任务分解、执行策略。

- 需求分析：理解用户的目的地、天数、预算和偏好
- 任务分解：将行程规划拆解为 5 个子任务，比如景点选择、路线优化、餐饮推荐等
- 执行策略：规划好先查什么、再查什么，确定执行顺序

这个阶段的输出是一份"工作计划"，它被传递给执行阶段，指导后续的工具调用如何展开。

第二阶段：EXECUTE（执行阶段）

这是 Agent 的核心循环。我们称之为 Agent Loop，最多运行 10 轮。每轮循环的执行流程如下：

第一步，调用 LLM。我们将累积的消息列表和工具定义一起发送到 SiliconFlow Chat Completions API。模型返回的响应中包含两个关键信息：tool_calls 数组和 usage 统计。

第二步，累积 Token 统计。每一次 LLM 调用后，我们都从响应中提取 usage 字段，累加到 GenerationStats 对象中。这个对象记录了总 Token 数、提示词 Token 数、生成 Token 数以及 API 调用次数。这样无论 Agent 循环多少轮，最终都能生成一份完整的 Token 消耗账单。

第三步，执行工具调用。如果 LLM 返回了 tool_calls，我们逐一解析执行。每个工具调用包含工具名称和参数，我们通过 mcpClient.callTool 调用高德 MCP Server。工具执行失败时最多重试 2 次。每次工具调用结束后，会记录一条 ToolCallRecord，包含工具名称、中文显示名和状态。

第四步，处理边界情况。如果 LLM 没有返回 tool_calls，有以下几种处理方式：

- 如果 finish_reason 是 "length"，说明输出被 max_tokens 截断了，使用更大的 max_tokens 重试
- 如果输出中包含 final_output 标签，尝试从中解析 JSON 格式的行程方案
- 如果 JSON 解析失败，切换到流式模式，通过逐步累积字符来获取完整响应

第五步，达到最大轮数。如果 10 轮耗尽仍未得到有效结果，系统会强制调用流式模式尝试兜底。如果仍然失败，则回退到普通 LLM 调用，至少返回一个文本结果。

第三阶段：VERIFY（验证阶段）

执行完成后，进入验证阶段。大模型会对生成的行程进行六项检查：完整性、真实性、合理性、预算匹配、偏好覆盖、描述质量。验证结果以 JSON 格式返回，包含 status、issues 数组和 summary。

如果返回 FAIL，最多重试 2 次，要求 LLM 针对具体的 issues 逐一修正。如果修正成功且返回了 final_output，则从中提取修正后的行程方案。

每个阶段开始和结束时，后端都会通过 toolNotifier 回调函数向 SSE 管道发送 phase 事件，前端据此更新进度指示器。

三、MCP 工具集成

Agent 的工具能力来源于 MCP（Model Context Protocol）协议。我们在 AmapMcpClient 内部类中实现了 MCP 客户端。

连接流程如下：

1. 通过 SSE 连接到高德 MCP Server 的端点
2. 接收 endpoint 事件，从中解析出 JSON-RPC 的 POST 调用地址
3. 发送 initialize 请求完成握手
4. 调用 tools/list 获取可用工具列表，自动转换为 LLM function calling 格式
5. 执行时调用 tools/call 传入工具名和参数

这种架构的好处是工具与 Agent 逻辑完全解耦。如果需要增加新的数据源，只需要注册一个新的 MCP Server，Agent 不需要任何代码修改。

目前集成的 MCP 工具包括：天气查询、地点搜索、周边搜索、地点详情查询、驾车路线规划、步行路线规划、骑行路线规划、公交路线规划、测距、坐标解析、地址解析、IP 定位、地图展示链接生成、导航链接生成、打车链接生成。

此外，我们还内置了一个自定义工具 search_destination_info，预置了 12 个中国热门城市的旅行知识库，包括北京、上海、杭州、成都、西安、广州等，不依赖外部 API 即可提供目的地基础信息。

四、SSE 流式传输实现

为了实现实时展示 Agent 的思考过程，我们采用了 SSE 协议。后端使用 Spring Boot 的 SseEmitter，超时时间设置为 600 秒。

后端 Agent 在运行过程中，通过 toolNotifier 回调函数向 SSE 管道写入 6 种类型的事件：

事件名	触发时机	数据格式
phase	阶段开始或结束时	JSON，包含 type 和 message 字段
thought	Agent 思考过程中	纯文本
tool	工具调用或完成时	纯文本
result	行程生成完成时	JSON 格式的 ItineraryPlan
stats	result 事件之后	JSON 格式的 GenerationStats
error	发生错误时	纯文本错误描述

前端的实现方式值得一提。我们没有使用标准的 EventSource API，而是使用原生 fetch 加 ReadableStream 手动解析 SSE 协议。原因有两个：

第一，原生 EventSource 只支持 GET 请求，而我们的 SSE 端点需要 POST 请求来提交行程参数。

第二，手动解析可以更精细地控制事件处理和错误恢复。

解析逻辑如下：读取流式数据块，累积到 buffer，按换行符分割，解析 event 和 data 行，遇到空行时触发对应的回调函数。整个解析过程在 itinerary.ts 中实现，大约 80 行代码，非常轻量。

五、前端 Agent 面板设计与实现

前端的 Agent 面板在 BattleCustomizePlan.vue 中实现，是整个用户体验的核心触点。面板有三种视觉状态。

状态一：待机模式。

当用户尚未生成行程时，面板展示引导文案，内容为"填写左侧表单后点击生成 AI 方案，Agent 将在这里展示实时规划与思考过程"。这个状态帮助用户理解面板的作用和使用方式。

状态二：运行模式。

生成或修改过程中，面板展示三阶段进度指示器：①规划、②执行、③验证。每个步骤有独立的动画效果——运行中的步骤显示脉冲动画，已完成步骤显示绿色对勾和填充连线。

下方是实时日志流，按照三种类型分别渲染：

- phase 类型：蓝色左边框横幅，阶段结束时变为绿色
- thought 类型：紫色虚线边框气泡，显示 AI 的思考内容
- tool 类型：灰色工具调用行，显示当前正在调用的工具名称

日志流会自动滚动到底部，始终跟随最新的输出。

状态三：完成模式。

生成完成后，面板切换为摘要视图，展示以下内容：

- 行程摘要：模型名称、天数、路线项数、预算
- Token 消耗统计卡片：总计 Token、提示词 Token、生成 Token、API 调用次数
- 工具调用明细清单：每次工具调用的名称和成功或失败状态
- 生成过程回顾：精简版的日志列表

这三种状态的切换通过 Vue 的 v-if 条件渲染实现。统计数据通过三个 ref 变量管理：generationStats 记录最新生成的统计，refineStats 记录最新修改的统计，loadedStats 记录从数据库加载的历史统计。显示优先级为：最新生成优先于最新修改，最新修改优先于数据库加载。

六、智能合并策略

最后介绍一下 AI 修改行程时的智能合并策略，实现在 mergeRefinedPlan 方法中。

当用户通过自然语言要求 AI 修改行程时，AI 返回的是完整的新行程方案，而不是增量 diff。但如果直接替换整个方案，用户之前手动编辑的调整就会全部丢失。

我们的策略是这样做的：

第一步，判断 AI 意图。比较 AI 返回的 totalDays 与原始方案的差异。如果减少了，说明 AI 决定删除某一天；如果增加了，说明 AI 决定新增一天；如果不变，则说明只做项目级别的修改。

第二步，保留用户修改。以原始方案为基础，只移除 AI 明确删除的项目。删除判断通过项目名称匹配来实现，这样用户手动添加或修改的内容可以完整保留下来。

第三步，合并新增内容。AI 新增的项目追加到对应天的末尾，AI 新增的天追加到方案末尾。

第四步，天数重编号。最后对所有天重新编号，保证序号连续。

这样无论用户先手动调整再让 AI 修改，还是先 AI 修改再手动调整，都能获得合理的合并结果。

总结

总结一下，IntelliTravel Pioneer 的 AI Agent 有以下四个核心特点：

第一，通过三阶段循环实现了可靠的行程规划——先规划思路，再执行获取数据，最后验证修正。
第二，通过 MCP 协议集成了丰富的地图数据工具，工具与 Agent 逻辑完全解耦，扩展性强。
第三，通过 SSE 流式协议实现了思考过程的全透明展示，6 种事件类型覆盖了完整的生成链路。
第四，通过智能合并策略保证了人机协作的顺畅体验，用户的每次手动调整都不会被 AI 覆盖。

技术栈方面，后端是 Spring Boot 3 加 Java 21，前端是 Vue 3 加 TypeScript，大模型使用 Qwen3-8B，工具层使用高德 MCP Server。各层职责清晰，整体架构具备良好的可扩展性。

以上就是 IntelliTravel Pioneer AI Agent 应用实现架构的完整介绍，谢谢大家。